HW3: Sentiment Analysis

[TA] Cheng-Tsung Lee

Introduction

In this homework, we will explore the fascinating field of sentiment analysis using deep learning techniques. Specifically, we will focus on multi-class classification, where the goal is to predict each sentence in posts from social media as belonging to the label.



Dataset

The Social Media Dataset consists of tens of thousands of sentences in posts from social media, annotated with sentiment labels (i.e., positive, neutral, and negative). We will work with this dataset to train and evaluate our classification models.



Tasks

1. Data Preparation:
    - Load and preprocess the Social Media Dataset.
    - Split the data into training, validation, and testing sets.
    - Preprocess the datasets to obtain better results.

2. Model Architecture:
    - Design customized network for this homework.
    - The following additional architecture might help: 1. MLP 2. CNN

Tasks

3. Training and Evaluation:
    - Train the model using appropriate loss function.
    - Monitor training process and validate on the validation set.
    - Evaluate the model's performance using macro-f1 score.

4. Inference and Visualization:
    - Apply the trained model to unseen sentences from the testing set.
    - Calculate macro-f1 score throughout the validation set.
    - Visualize the outcome of both training and validation sets.

Learning Objectives

By completing this homework, we aim to:

- ✓ Gain hands-on experience with sentiment analysis tasks.
- ✓ Understand the challenges of natural language processing.
- ✓ Learn to interpret classification results and assess model performance.

Important Date

Start Date: 2026/11/10 (Tue.) 16:00

Source code / Checkpoint submission deadline on E3: 2026/11/30 (Mon.) 23:59

Model prediction submission deadline on Kaggle: 2026/11/30 (Mon.) 23:59

Submission Form

Please zip all files in one file and named “HW3_{student_id}.zip” when uploading to E3 platform.

We will also have submission on **“Kaggle”.** Please make sure the team’s name is your student ID, and the account is linked to the email provided on the class participant list. Each account can submit the answer 3 times every 24 hours.

Kaggle link: https://www.kaggle.com/t/89f6224e259745938f459f25926f095c

Requirements

1. Design your own data preprocessing method.
2. Construct any training pipelines on your own.
3. You can ONLY use **“google-bert/bert-base-uncased”** as your base model.
4. Plot each epoch’s training phase and validating phase to observe how they change.
5. If needed, an ensemble of up to three models can be used.

Dataset Files

The dataset includes two primary files:

“dataset.csv”: This file includes 9,001 csv rows, where there are a header and 9,000 rows of data represented as follows: “id” – the index of data; “text” – sentence in string format; “label” – sentiment label in string format.

“benchmark.csv”: This file includes 1000 unseen questions without answer. This is used to measure the final performance of your models and training pipelines.

File Structure

Please organize your project files using the following directory structure that separates data, code, and other resources.

```
HW3_{student_id}/
├── dataset/
│   ├── benchmark.csv
│   └── dataset.csv
├── saved_models/
│   └── checkpoint
│       └── ...
├── main.py
├── README.md
└── requirements.txt
```

Marco-F1 score

It is a metric used to evaluate the performance of a classification model. It calculates the F1-score for each class separately and then takes the average across all classes.

First, calculate Precision and Recall for each class:

Precision:

$$Precision_i = \frac{TP_i}{TP_i + FP_i}$$

Recall:

$$Recall_i = \frac{TP_i}{TP_i + FN_i}$$

Then calculate the F1-score for each class:

$$F1_i = 2 \times \frac{Precision_i \times Recall_i}{Precision_i + Recall_i}$$

Finally, calculate the average F1-score across all classes:

$$\boxed{Macro\text{-}F1 = \frac{1}{K} \sum_{i=1}^{K} F1_i}$$

Where K is the total number of classes.

Grading Policy

Homework score = Experimental results (100%)

Experimental results (100%) (Note that result depends on benchmark set)

[100 pts]: macro-f1 score ≥ 92%

[90 pts]: macro-f1 score ≥ 90.5%

[80 pts]: macro-f1 score ≥ 89%

[0 pts]: macro-f1 score < 89%

Notice: If the zip filename has a format error, it will be a penalty (-10 pts).

Rules

Please only train & validate your model on the training & validating dataset split by the provided dataset. If the TA found your model trained on the benchmark set or any other additional datasets through retraining your model, you'll get 0 pts in this homework.

Please ensure the TA can execute the code you uploaded. If the TA fails to do so, you'll get 0 pts in this homework.

If you're found faking your model predicting results, you'll get 0 pts in this homework. Also, we will take disciplinary action under school regulations.

Plagiarism is strictly prohibited. If you're found guilty of plagiarizing another's code or results, you and the person/people you plagiarized with will get 0 pts in this homework. Also, we will take disciplinary action under school regulations.

Rules

If any issues are identified in your submitted files for this lab, the TA will reach out to you via email. Please monitor your inbox closely. Suppose the TA has not received your response within three days of sending the email. In that case, it will be considered that you have forfeited the opportunity to provide further clarification and have accepted the corresponding penalty or consequences.

Contact & Information

Please post your question on the E3 forum.

[TA] Cheng-Tsung Lee (李承璁): ctlee.ee14@nycu.edu.tw

[TA hours] 13:00-15:00 Wed. ED-716 (Please make an appointment by email first)

Hands-on Tutorials on NLP

Not code implementation anymore!

Outline

1. **What you need to know about your data!** – *EDA (Exploratory Data Analysis)*
2. **Is your data ready for training?** – *Data Preprocessing*
3. **Dealing with imbalanced data.** – *Class Imbalance Handling*
4. **Train your model more effectively!** – *Model Training Tips*
5. **How to make your model more robust?** – *Adversarial Training*
6. **Can we make better predictions with multiple models?** – *Ensemble Learning*

EDA (Exploratory Data Analysis)

Before training your model, it's important to understand your dataset. Since we are working with tweets, here are some key aspects worth exploring:

1. Class Distribution:
    - Analyze the number and percentage of samples in each class.
    - Check whether the dataset is balanced or if certain classes are underrepresented.

2. Token Length Distribution:
    - Analyze the distribution of tokenized tweet lengths (e.g., mean, median, percentiles).
    - Use these statistics to determine an appropriate max sequence length, balancing information retention and computational efficiency.

EDA (Exploratory Data Analysis)

3. Text Patterns:
    - Examine the frequency of common Twitter-specific elements, such as emojis, URLs, hashtags, and mentions (@username).
    - Consider whether these elements provide useful information or require special handling during preprocessing.

Data Preprocessing

Before feeding tweets into your model, it's crucial to preprocess the text to reduce noise and improve data consistency. Here are some common preprocessing techniques worth exploring:

1. URL Normalization:
    - Replace all URLs with a special token (e.g., [URL]) instead of keeping the original links.
    - Preserve the presence of URLs while reducing unnecessary variations in the text.

2. URL Removal:
    - Remove URLs entirely if they do not provide useful information for the task.
    - Reduce noise and prevent the model from learning irrelevant URL patterns.

Data Preprocessing

3. Emoji-to-Text Conversion:
   - Convert emojis into their textual descriptions (e.g., 😊 → smiling_face).
   - Preserve emotional and semantic information in a text-based format.

4. Emoji Removal:
   - Remove emojis if they are considered irrelevant or noisy.
   - Simplify the input text but be aware that removing emojis may discard useful sentiment information.

Class Imbalance Handling

In real-world datasets, some classes may contain significantly more samples than others. This imbalance can cause the model to favor majority classes while performing poorly on minority classes. Here are some techniques worth exploring:

1. Weighted Cross-Entropy Loss:
    - Assign higher loss weights to minority classes and lower weights to majority classes.
    - Encourage the model to pay more attention to underrepresented classes during training.

Class Imbalance Handling

2. Balanced Sampling:
    - Adjust the sampling probabilities to ensure minority classes are sampled more frequently during training.
    - Reduce the model's bias toward majority classes by creating more balanced training batches.

Model Training Tips

Training a good model requires more than just choosing the right architecture. Proper training strategies and hyperparameter tuning can improve model performance, stability, and generalization. Here are some techniques worth exploring:

1. Learning Rate Scheduling (Linear / Cosine Warmup):
    - Gradually increase the learning rate during the warmup phase to stabilize early training.
    - Apply linear or cosine learning rate decay to improve convergence throughout training.

2. Early Stopping:
    - Monitor validation performance and stop training when the model stops improving.
    - Prevent overfitting and avoid unnecessary training epochs.

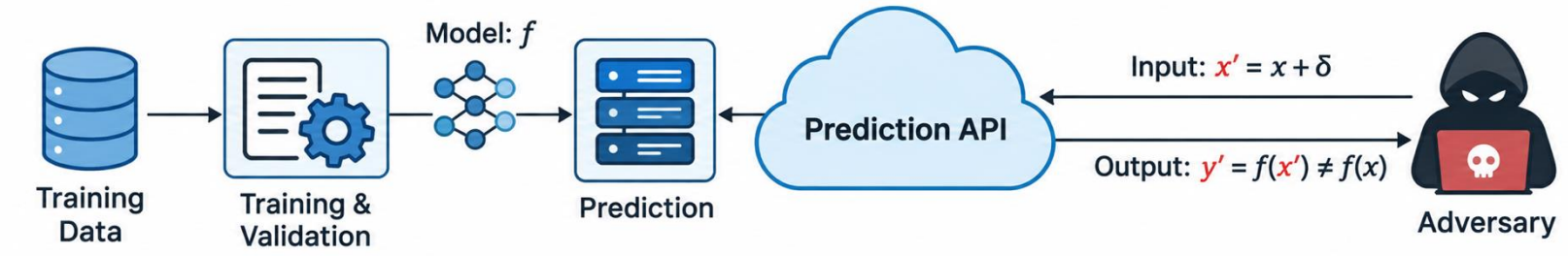
Model Training Tips

3. Grid Search:
    - Experiment with different combinations of hyperparameters (e.g., learning rate, batch size, weight decay).
    - Identify the best hyperparameter configuration based on validation performance.

4. Adding an Additional Classification Head:
    - Add one or more fully connected layers (MLP) on top of BERT's output before the final classification layer.
    - Explore whether a more expressive classification head improves performance compared to a simple linear classifier.

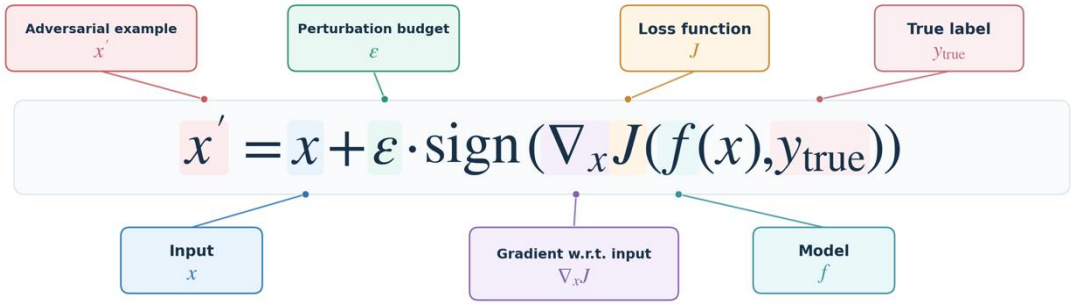
Adversarial Training

Adversarial training is a technique that improves model robustness by introducing small, carefully designed perturbations during training. These perturbations challenge the model to make more stable predictions and generalize better to unseen data.



Adversarial Training

1. Fast Gradient Sign Method (FGSM):
   - Generate adversarial perturbations using the gradients of the loss function.
   - For text classification, apply small perturbations to word embeddings rather than directly modifying discrete tokens.
   - Train the model using both original and adversarial examples to encourage more robust predictions.



Adversarial Training

2. Adversarial Training Strategies:
    - Experiment with different perturbation strengths (e.g., epsilon values) to control the magnitude of adversarial noise.
    - Compare model performance with and without adversarial training to evaluate its effectiveness.

Ensemble Learning

Ensemble learning is a technique that combines predictions from multiple models to improve overall performance, stability, and generalization. Instead of relying on a single model, ensemble methods leverage the strengths of different models to produce more reliable predictions.

1. Homogeneous Ensembles:
    - Train multiple models with the same architecture (e.g., BERT) but different random seeds, hyperparameters, or training configurations.
    - Combine their predictions to reduce variance and improve overall model performance.

Ensemble Learning

2. Soft / Hard Voting:
    - Soft Voting – Average the predicted class probabilities from multiple models and select the class with the highest average probability.
    - Hard Voting – Each model predicts a class label, and the final prediction is determined by majority voting.
    - Compare both voting strategies to determine which produces better classification results.

Ensemble Learning

